Gradient Boosting, GBDT, XGBoost, LightGBM

Simon Liu

Oct 29, 2026

Announcement

Quiz 3

- Next Thursday, Nov 5th, at 8pm.
- Duration: 50mins.
- Similar structure to Quiz 1 and 2, online, in-class, lock-down browser.
- True/false, multiple choice, short calculations.
- Coverage: Lecture content from Week 9 to 11.
- Python notebook content not covered.

Review

Review: Random Forest

- Ensemble of decision trees built via **bagging** (bootstrap sampling)
- Each tree trained on a different bootstrap sample; splits consider only a random subset of features $\rightarrow$ reduces inter-tree correlation
- Final prediction by **majority vote** (classification) or **average** (regression)
- Main benefit: **variance reduction** with minimal change in bias

Review: Boosting (General Concept)

- Sequential ensemble method: each weak learner corrects errors of its predecessor
- Combines weak learners $h_m(x)$ into a strong model via weighted sum:

$$F(x) = \sum_{m=1}^{M} \alpha_m \, h_m(x)$$

- Focuses learning on "hard" examples by adjusting instance weights (or fitting residuals)
- Primary effect: reduces **bias** (and often variance) by iterative refinement

Review: AdaBoost

- Adaptive boosting algorithm that re-weights training examples each round
- Misclassified instances gain higher weight $\rightarrow$ subsequent weak learners $h_m(x)$ focus on them
- Learner weight:

$$\alpha_m = \frac{1}{2} \ln\left(\frac{1-\epsilon_m}{\epsilon_m}\right)$$

- Instance weight update (normalized):

$$w_i \leftarrow w_i \, \exp\big(-\alpha_m \, y_i \, h_m(x_i)\big)$$

- Final prediction by weighted vote; excellent bias reduction but sensitive to noisy data

Gradient Boosting

Gradient Boosting: General Concept

- Ensemble method that builds models **sequentially**, each correcting errors of its predecessor
- Weak learners (typically shallow decision trees) are added one at a time
- Each new learner is fitted to the **residual errors** (differences between true targets and current model predictions)
- Final model is an additive combination of weak learners:

$$F(x) = \sum_{m=1}^{M} \gamma_m \, h_m(x)$$

- Key benefit: systematically reduces **bias** while controlling variance

Loss Functions & Pseudo-Residuals

- Gradient boosting minimizes a **differentiable loss function** $L(y, F(x))$
- At iteration m, compute **pseudo-residuals** as negative gradients of the loss:

$$r_i^{(m)} = -\frac{\partial L(y_i, F^{(m-1)}(x_i))}{\partial F^{(m-1)}(x_i)}$$

- Pseudo-residuals represent the direction of steepest descent in loss space
- Weak learner $h_m(x)$ is trained to predict these residuals

Functional Gradient Descent in Function Space

- Gradient boosting performs **gradient descent in function space**, not parameter space
- Each added weak learner approximates the negative gradient of the loss function
- Update rule at iteration m:

$$F^{(m)}(x) = F^{(m-1)}(x) + \gamma_m \, h_m(x)$$

- Step size γ_m chosen to minimize overall loss:

$$\gamma_m = \arg\min_{\gamma} \sum_{i=1}^{N} L\big(y_i,\ F^{(m-1)}(x_i) + \gamma \, h_m(x_i)\big)$$

- Enables efficient optimization of complex, non-linear models

What Is a Functional Gradient?

- Ordinary gradient descent takes derivatives w.r.t. model **parameters**
- Functional gradient descent takes derivatives w.r.t. the **model function** itself
- The functional derivative $\frac{\delta L}{\delta F}(x)$ measures how a small change in F at x affects total loss
- Intuition: it points to the direction in "function space" that most rapidly decreases loss
- Weak learners serve as basis directions; the functional gradient identifies which function to add next

Computing Functional Gradients in Practice

- Exact functional gradients are infinite-dimensional; we approximate them via **pseudo-residuals**
- At iteration m, for each example (x_i, y_i):

$$r_i^{(m)} = -\frac{\partial L\big(y_i,\, F^{(m-1)}(x_i)\big)}{\partial F^{(m-1)}(x_i)}$$

- Pseudo-residuals become the numeric targets for the next weak learner $h_m(x)$
- Training h_m on $\{(x_i, r_i^{(m)})\}$ finds the direction that decreases loss fastest

Squared Error Example

Loss: $L(y, F(x)) = (y - F(x))^2$

Derivative w.r.t. F:

$$\frac{\partial L}{\partial F} = -2\,(y - F)$$

Pseudo-residual:

$$r = -\frac{\partial L}{\partial F} = 2\,(y - F)$$

Numeric example: $y = 5,\ F = 3 \quad \rightarrow \quad r = 2 \times (5 - 3) = 4$

Logistic Loss Example

Loss (binary): $L(y, F) = \ln\left(1 + e^{-yF}\right),\ y \in \{-1, +1\}$

Derivative w.r.t. F:

$$\frac{\partial L}{\partial F} = -\frac{y}{1 + e^{yF}}$$

Pseudo-residual:

$$r = -\frac{\partial L}{\partial F} = \frac{y}{1 + e^{yF}}$$

Numeric examples:

$y = +1,\ F = 0 \rightarrow r = 0.5$
$y = -1,\ F = 1 \rightarrow r \approx -0.27$

Introduction to Gradient-Boosted Trees

- A specialized form of boosting that uses decision trees as base learners
- First formalized by Friedman (1999) to combine gradient descent with tree fitting
- Key advantages over generic boosting:
  - Naturally handles non-linear feature interactions
  - Offers built-in feature selection via split importance
  - Highly customizable via choice of loss function and regularization
- Widely used in practice for both regression and classification tasks

Gradient-Boosted Trees: Core Algorithm (Part 1)

1. Initialize model with constant prediction:

$$F_0(x) = \arg\min_{\gamma} \sum_{i=1}^{N} L(y_i, \gamma)$$

2. For $m = 1 \ldots M$:
   - Compute pseudo-residuals:

$$r_i^{(m)} = -\frac{\partial L(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)}$$

   - Fit a regression tree $h_m(x)$ to $\{(x_i, r_i^{(m)})\}_{i=1}^{N}$

Gradient-Boosted Trees: Core Algorithm (Part 2)

3. Determine optimal step-size γ_m:

$$\gamma_m = \arg\min_{\gamma} \sum_{i=1}^{N} L\big(y_i,\ F_{m-1}(x_i) + \gamma\, h_m(x_i)\big)$$

4. Update the model:

$$F_m(x) = F_{m-1}(x) + \gamma_m\, h_m(x)$$

5. Final ensemble after M iterations: $F_M(x)$

Decision Trees as Weak Learners

- Each weak learner is a shallow regression tree (typical depth 3–5)
- Trees partition feature space via binary splits; each leaf outputs a constant prediction
- Strengths of decision trees:
  - Capture complex, non-linear relationships
  - Handle both categorical and numerical features without scaling
  - Provide interpretable split importance scores
- Aggregating many simple trees balances bias and variance, yielding robust predictions

Recap: Core GBT Algorithm & Global Line Search

- Initialize $F_0(x)$ to minimize total loss
- Each iteration m:
  1. Compute pseudo-residuals
  2. Fit regression tree $h_m(x)$
  3. Perform global line search for γ_m:

$$\gamma_m = \arg\min_{\gamma} \sum_{i=1}^{N} L\big(y_i, F_{m-1}(x_i) + \gamma\, h_m(x_i)\big)$$

  4. Update model: $F_m(x) = F_{m-1}(x) + \gamma_m\, h_m(x)$

Global Line Search (Squared Loss)

- Loss: $L(y, F) = (y - F)^2$
- Objective:

$$\sum_{i=1}^{N} \left(y_i - [F_{m-1}(x_i) + \gamma\, h_m(x_i)]\right)^2$$

- Differentiate w.r.t. γ, set to zero:

$$\gamma_m = \frac{\sum_i h_m(x_i)\,(y_i - F_{m-1}(x_i))}{\sum_i [h_m(x_i)]^2}$$

Global Line Search Example

- Predictions $F_{m-1} = \{3, 4, 5\}$, targets $\{5, 6, 8\}$, tree outputs $h_m = \{1, 1, 1\}$
- Numerator $= (5 - 3) + (6 - 4) + (8 - 5) = 7$
- Denominator $= 1^2 + 1^2 + 1^2 = 3$
- $\gamma_m = 7/3 \approx 2.33$

Introducing Region-Specific Step Sizes

- Instead of one global γ_m, compute one $\gamma_{m,r}$ per leaf (region)
- Each leaf's update optimally reduces loss for just that partition
- Conceptually: perform separate line search on each leaf's subset of data

	Global γ_m	**Region-Specific $\gamma_{m,r}$**
Scale	Single scalar for entire tree	One scalar per leaf
Flexibility	Uniform adjustment (coarse)	Tailored update per region
Loss Reduction	Good overall reduction	Greater local reduction
Computational Cost	Low (one line search)	Higher (one per leaf)

Region-Specific Line Search (Squared Loss)

For leaf r, minimize the sum of squared errors over only the samples in that leaf:

$$J(\gamma) = \sum_{i \in r} \big(y_i - [F_{m-1}(x_i) + \gamma]\big)^2,$$

where $|r|$ is the number of training examples in leaf r. Differentiate w.r.t. γ and set to zero:

$$\frac{dJ}{d\gamma} = -2 \sum_{i \in r} \big(y_i - F_{m-1}(x_i) - \gamma\big) = 0.$$

Solve for the optimal update:

$$\gamma_{m,r} = \frac{\sum_{i \in r} \big(y_i - F_{m-1}(x_i)\big)}{|r|}.$$

Region-Specific Example

- Leaf r contains:
  - $x_1: \ F = 3, \ y = 5$
  - $x_2: \ F = 4, \ y = 7$
  - $x_3: \ F = 5, \ y = 9$
- Numerator $= (5 - 3) + (7 - 4) + (9 - 5) = 9$
- $|r| = 3 \rightarrow \gamma_{m,r} = 9/3 = 3$

Shrinkage (Learning Rate)

- Adds a multiplicative factor $0 < \eta \leq 1$ to each tree's update:

$$F_m(x) = F_{m-1}(x) + \eta\, \gamma_m\, h_m(x)$$

- Smaller $\eta \rightarrow$ slower learning $\rightarrow$ smoother convergence $\rightarrow$ reduces overfitting
- Typical values: $\mathbf{0.01}$–$\mathbf{0.1}$
- Trade-off: lower η requires more trees but improves generalization

Subsampling: Rows & Columns

- **Row subsampling** (stochastic boosting): train each tree on a random subset of training examples (50–80%)
  - Reduces variance and speeds up training
- **Column subsampling**: randomly select subset of features at each split (50–80%) or per tree
  - Lowers correlation between trees $\rightarrow$ improved robustness
- Combined effect: introduces randomness, mitigates overfitting, and enhances scalability

Tree Structure Hyperparameters

Parameter	**Purpose**	**Typical Range**
Number of trees	Controls ensemble size	100–1000
Max depth	Controls tree complexity	3–7
Min samples to split	Minimum examples to split a node	2–10
Min samples per leaf	Prevents tiny leaves	5–20
Max leaves	Limits total leaves in tree	20–100

Practical Tuning Guide & Early Stopping

- **Learning rate (η)**: 0.01–0.1
- **Number of trees**: 100–1000 (increase if η is small)
- **Subsampling**: rows 0.6–0.8; columns 0.5–0.8
- **Tree depth**: 3–7
- **Early stopping**: monitor validation loss; stop after 20–50 rounds without improvement
- Compare training vs validation error to detect overfitting

Bias & Variance in Gradient Boosted Trees

- Gradient boosting sequentially fits trees to residual errors $\rightarrow$ dramatically reduces **bias**
- Each added tree corrects mistakes of previous ensemble, driving predictions closer to true values
- Without regularization, boosting can increase **variance** by overfitting noise
- Variance control mechanisms:
  - **Shrinkage (learning rate)** slows updates $\rightarrow$ smoother learning
  - **Row subsampling** introduces randomness $\rightarrow$ reduces correlation between trees
  - **Column subsampling** limits features per split $\rightarrow$ lowers overfitting
  - **Tree complexity constraints** (depth, min samples) prevent overly specific leaves
- Net effect: large reduction in bias with moderated variance $\rightarrow$ improved generalization

Comparison: AdaBoost vs Gradient Boosted Trees

Characteristic	**AdaBoost**	**Gradient Boosted Trees**
Base Learner	Decision stump (depth=1)	Shallow regression tree (depth 3–7)
Error Correction	Reweights misclassified instances	Fits pseudo-residuals via gradient descent
Step Size	Learner weight α_m from error rate	Global γ_m via line search
Loss Function	Exponential loss (classification only)	Any differentiable loss (regression & classification)
Regularization	Implicit via simple weak learners	Explicit via shrinkage, subsampling, tree constraints
Overfitting Risk	Higher with noisy data	Reduced with learning rate & subsampling
Hyperparameters	Iterations, stump complexity	Trees, depth, learning rate, subsample rates

Gradient Boosting & Gradient-Boosted Trees: Summary

- Gradient boosting builds a strong model by sequentially adding weak learners that correct previous errors via gradient descent in function space
- Each iteration fits a new tree to the pseudo-residuals of the current ensemble prediction
- Updates are scaled by a step-size (γ) and optionally region-specific adjustments for finer control
- Key enhancements in GBT: shrinkage (learning rate), row & column subsampling, and tree structure constraints
- Allows flexible loss functions for both regression and classification
- Balances bias reduction (via sequential learning) with variance control (via regularization) for high predictive accuracy

Coding Sample 1

Break

XGBoost

Introduction to XGBoost

- XGBoost (eXtreme Gradient Boosting): a highly optimized implementation of gradient boosted trees
- Enhances standard GBT with:
  - Explicit regularization (split complexity & leaf weight penalties)
  - Second-order gradient optimization (uses Hessians)
  - Sparse-aware split finding (handles missing values natively)
  - Cache-efficient, parallelized training for large datasets
- Widely used in practice for its speed, accuracy, and scalability

XGBoost Objective Function

$$J = \sum_{i=1}^{N} L(y_i, \hat{y}_i) + \sum_{m=1}^{M} \Omega(f_m),$$

$$\Omega(f_m) = \gamma T + \tfrac{1}{2}\lambda \sum_{j=1}^{T} w_j^2,$$

- L: differentiable loss (e.g., squared error, logistic)
- T: number of leaves in tree f_m
- w_j: weight output at leaf j
- γ: penalty for adding a leaf (controls tree complexity)
- λ: L2 regularization on leaf weights (reduces overfitting)

Taylor Expansion: Objective for New Tree

Starting from the objective when adding tree f_m:

$$\sum_{i=1}^{N} L\big(y_i,\ F_{m-1}(x_i) + f_m(x_i)\big).$$

Perform a second-order Taylor expansion of L around $F_{m-1}(x_i)$:

$$L(y_i, F_{m-1} + f_m) \approx L(y_i, F_{m-1}) + \frac{\partial L}{\partial \hat{y}}\Big|_{F_{m-1}} f_m(x_i) + \frac{1}{2}\frac{\partial^2 L}{\partial \hat{y}^2}\Big|_{F_{m-1}} [f_m(x_i)]^2.$$

Taylor Expansion: Gradient & Hessian Terms

Define the pseudo-gradients:

$$g_i = \left.\frac{\partial L(y_i, \hat{y})}{\partial \hat{y}}\right|_{\hat{y}=F_{m-1}(x_i)}, \quad h_i = \left.\frac{\partial^2 L(y_i, \hat{y})}{\partial \hat{y}^2}\right|_{\hat{y}=F_{m-1}(x_i)}.$$

Summing across all samples gives the approximate loss change:

$$\sum_{i=1}^{N} \left[g_i f_m(x_i) + \tfrac{1}{2} h_i f_m(x_i)^2 \right].$$

Taylor Expansion: Full Approximate Objective

Adding XGBoost's regularization term $\Omega(f_m) = \gamma T + \frac{1}{2}\lambda \sum_j w_j^2$:

$$\sum_{i=1}^{N} \left[g_i f_m(x_i) + \frac{1}{2} h_i f_m(x_i)^2 \right] + \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2.$$

This is the function we optimize when constructing each new tree.

Leaf Objective

Focus on a single leaf j, where $f_m(x_i) = w_j$ for $i \in I_j$:

$$J_j(w_j) = \sum_{i \in I_j} \left[g_i w_j + \frac{1}{2} h_i w_j^2 \right] + \frac{1}{2} \lambda w_j^2.$$

Compute derivative of $J_j(w_j)$ w.r.t. w_j:

$$\frac{dJ_j}{dw_j} = \sum_{i \in I_j} g_i + w_j \left(\sum_{i \in I_j} h_i + \lambda \right).$$

Set derivative equal to zero for optimality.

Optimal Leaf Weight

Solve for w_j^*:

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}.$$

Interpretation:

- Numerator aggregates gradient direction in region
- Denominator scales update by curvature and regularization

Example: Optimal Leaf Weight (Squared Loss)

Squared loss: $L(y, \hat{y}) = (y - \hat{y})^2$

$$g_i = \frac{\partial L}{\partial \hat{y}} = -2(y_i - F_{m-1}(x_i)), \quad h_i = \frac{\partial^2 L}{\partial \hat{y}^2} = 2$$

For leaf j containing three samples:

i	y_i	$F_{m-1}(x_i)$	g_i
1	5	3	$-2(5-3) = -4$
2	7	4	$-2(7-4) = -6$
3	9	5	$-2(9-5) = -8$

Compute sums:

$$\sum_i g_i = -18, \quad \sum_i h_i = 3 \times 2 = 6.$$

With $\lambda = 0$, the optimal leaf weight:

$$w_j^* = -\frac{-18}{6+0} = 3.$$

Effect of Regularization (λ) on Leaf Weight

Using the same three-sample leaf ($\sum g_i = -18, \sum h_i = 6$), the optimal weight is

$$w_j^* = -\frac{\sum_i g_i}{\sum_i h_i + \lambda}.$$

λ	Denominator $6 + \lambda$	w_j^*	Intuition
0	6	3.00	No regularization $\rightarrow$ full residual correction
2	8	2.25	Moderate penalty $\rightarrow$ shrinks update toward zero
10	16	1.125	Strong penalty $\rightarrow$ much smaller correction, reduces overfitting

- Increasing λ applies stronger L2 regularization on leaf weights
- Larger λ forces leaf predictions closer to zero, damping extreme updates
- Helps prevent overfitting by limiting how much each tree can correct residual errors

Tree Pruning & Complexity Penalty (γ)

- XGBoost adds a constant penalty γ for each additional leaf split
- A split is only performed if it yields a net reduction in the approximate objective greater than γ
- Prevents creation of splits that offer only marginal improvement, controlling tree complexity
- Balances reduction in loss against the cost of increasing model size $\rightarrow$ reduces overfitting

Split Gain Formula with γ

For a candidate split dividing parent node into left (L) and right (R) leaves:

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma$$

- $G_L = \sum_{i\in L} g_i$, $H_L = \sum_{i\in L} h_i$ (similarly for R)
- λ is leaf weight regularization
- Split accepted only if Gain > 0
- Higher $\gamma \rightarrow$ stricter requirement for splitting $\rightarrow$ simpler trees

Numeric Example of Gain Calculation

Leaf statistics:

$$G_L = 10, \quad H_L = 8; \quad G_R = -4, \quad H_R = 5; \quad \lambda = 0$$

Pre-penalty gain calculation:

$$\frac{1}{2}\left[\frac{10^2}{8} + \frac{(-4)^2}{5} - \frac{6^2}{13}\right] \approx 6.5$$

- If $\gamma = 5$, final Gain $6.5 - 5 = 1.5 \rightarrow$ split accepted
- If $\gamma = 7$, final Gain $6.5 - 7 = -0.5 \rightarrow$ split rejected

Effect of γ on Tree Structure

γ	Approx. Tree Depth	# Leaves	Overfitting Risk
0	Deep	Many	High
5	Moderate	Moderate	Moderate
10	Shallow	Few	Low

- Increasing γ prunes weak splits, producing simpler trees
- Controls variance by limiting tree complexity without significantly increasing bias
- Key tuning knob for balancing model accuracy and generalization

Missing-Value Handling in XGBoost

- XGBoost treats missing values as a separate category; no imputation needed
- During split evaluation, it learns a **default direction** (left or right) for missing entries
- For each candidate split, XGBoost computes gain twice:
  - Once sending missing values left
  - Once sending missing values right
- Chooses the direction that yields higher gain $\rightarrow$ automatically handles sparsity
- Default direction is used consistently at training and inference

Missing-Value Handling: Numeric Example

Sample	Feature X
1	8
2	12
3	— (missing)
4	5
5	— (missing)

Candidate split: $X < 10$

Missing to	Gain
Left	4.2
Right	1.8

Default direction: Left (higher gain) $\rightarrow$ both missing values routed to left child

Cache-Aware Block Structure in XGBoost

- **Columnar Block Storage**: data stored in compressed column blocks to maximize cache locality
- **Histogram-Based Split Finding**: builds fixed-width histograms per feature to reduce memory scans
- **Cache Line Optimization**: processes data in contiguous blocks sized to fit CPU cache lines, minimizing cache misses
- **Reuse of Computation**: once histograms are built for a block, they are reused for multiple splits without reloading raw data
- **Out-of-Core Support**: spills blocks to disk when data exceeds RAM, streaming in cache-friendly batches

Parallel & Distributed Learning in XGBoost

- **Multi-Threaded Histogram Construction**: threads build feature histograms in parallel then merge results
- **Column Parallelism**: simultaneous split evaluation across different features
- **Data Parallelism (Distributed)**: each node computes local histograms; global aggregation via AllReduce
- **GPU Acceleration**: histogram generation and split gain calculations executed on GPU for massive speedups
- **Efficient Synchronization**: lightweight communication minimizes network overhead during distributed training
- **Elastic Scaling**: supports dynamic addition/removal of worker nodes without restarting training

XGBoost Training Algorithm (Part 1)

1. **Initialize** model output:

$$F_0(x) = \arg\min_{\gamma} \sum_{i=1}^{N} L(y_i, \gamma)$$

2. **For** m = 1 . . . M:
   - Compute gradients and Hessians:

$$g_i = \frac{\partial L(y_i, F_{m-1}(x_i))}{\partial F}, \quad h_i = \frac{\partial^2 L(y_i, F_{m-1}(x_i))}{\partial F^2}$$

   - Optional row and column subsampling
   - **Build tree** $f_m(x)$ (leaf-wise growth):
     - Evaluate every candidate split's gain:

$$\text{Gain} = \frac{1}{2}\Big[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}\Big] - \gamma$$

     - Accept split only if Gain > 0 (pre-pruning)
     - Stop splitting when no valid gain, or max depth/min child size reached

XGBoost Training Algorithm (Part 2)

3. Compute optimal leaf weight for each terminal node j:

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$$

4. Update ensemble prediction:

$$F_m(x) = F_{m-1}(x) + \eta \, f_m(x)$$

5. Repeat until M trees are built
6. **Return** final model $F_M(x)$

Key Hyperparameters & Tuning for XGBoost

Parameter	**Description**	**Typical Range / Guidance**
Learning rate (η)	Step size shrinkage	0.01 – 0.2 (lower $\rightarrow$ better generalization, slower)
Number of trees	Number of boosting rounds	100 – 1000 (use early stopping)
Max depth	Maximum tree depth	3 – 7 (deeper $\rightarrow$ more complexity)
Subsample	Row sampling ratio per tree	0.6 – 0.8 (reduces overfitting)
Colsample_bytree	Column sampling ratio per tree	0.5 – 0.8 (reduces feature correlation)
Gamma (γ)	Minimum split gain	0 – 5 (higher $\rightarrow$ stricter split)
Lambda (λ)	L2 regularization on leaf weights	1 – 10 (higher $\rightarrow$ smaller leaf weights)
Min_child_weight	Minimum sum of instance weights in leaf	1 – 10 (higher $\rightarrow$ prevents small leaves)

XGBoost Benchmarks & Real-World Impact

- Widely adopted: used in over 70% of winning Kaggle solutions
- Performance: 5×–10× faster training than traditional GBT implementations on large tabular datasets
- Accuracy: consistently ranks among top methods for regression and classification tasks, often reducing test error by 10–30
- Industry use cases:
  - Credit scoring at leading financial institutions
  - Click-through rate prediction at major ad platforms
  - Fraud detection, customer churn prediction, and demand forecasting
- Scalability: handles datasets with tens of millions of rows using distributed training (AllReduce) and GPU acceleration

LightGBM

Introduction to LightGBM

- LightGBM (Light Gradient Boosting Machine) by Microsoft (Ke et al., 2017)
- Designed for extreme speed and efficiency on large, high-dimensional datasets
- Key innovations:
  - Leaf-wise (best-first) tree growth
  - Gradient-based One-Side Sampling (GOSS)
  - Exclusive Feature Bundling (EFB)
- Widely adopted in industry for real-time prediction and large-scale modeling

LightGBM Objective Function

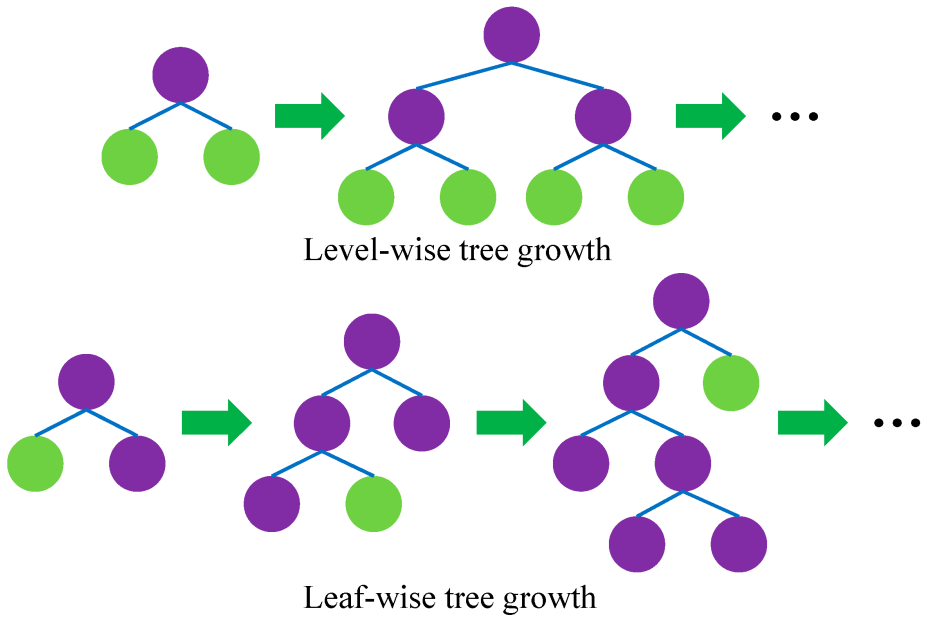
Minimize regularized objective:

$$J = \sum_{i=1}^{N} L(y_i, \hat{y}_i) + \sum_{m=1}^{M} \left(\gamma T + \tfrac{1}{2} \lambda \sum_{j=1}^{T} w_j^2 \right)$$

- L: differentiable loss (e.g., squared error, logistic)
- T: number of leaves in tree m
- w_j: leaf weight; γ penalizes complexity; λ penalizes large weights
- Optimization via second-order Taylor expansion (gradients & Hessians)

Leaf-Wise (Best-First) Tree Growth

- Traditional GBT grows trees level by level; LightGBM grows the leaf with highest loss reduction first
- At each iteration, select the single leaf with maximum split gain to split — leads to deeper, more focused trees
- Pros: achieves greater reduction in loss per split $\rightarrow$ fewer trees required for same accuracy
- Cons: can overfit small datasets if not controlled (mitigated by hyperparameters like max_depth, num_leaves)



Why LightGBM Trees Differ (num_leaves & Stopping Criteria)

- LightGBM grows trees **leaf-wise** (best-first), not level-wise
- Splitting continues until the total number of leaves reaches **num_leaves**, rather than expanding all branches equally
- Additional stopping controls:
  - **min_data_in_leaf**: prevents splits on very small partitions
  - **min_gain_to_split**: ignores splits with insufficient gain
- Result: LightGBM allocates capacity to the highest-error regions, producing deeper, narrower trees; level-wise trees remain more balanced

Numeric Example: Level-Wise Growth (num_leaves=4)

Split gains:

Node	Gain
Root	20
Left	12
Right	4
Left→Left	8

- Split Root → Left & Right (gain 20)
- Depth 2: split both Left (12) and Right (4) → yields 4 leaves
- Stop (num_leaves=4 reached)

Numeric Example: Leaf-Wise Growth (num_leaves=4)

Same split gains:

Node	Gain
Root	20
Left	12
Left→Left	8
Left→Left→Left	6

- Split Root → Left & Right
- Split highest-gain leaf (Left, gain 12)
- Next split highest-gain leaf (Left→Left, gain 8)
- Next split highest-gain leaf (Left→Left→Left, gain 6) → yields 4 leaves
- Stop (num_leaves=4 reached)

Gradient-Based One-Side Sampling (GOSS): Concept

- GOSS accelerates boosting by focusing on the most informative examples (largest gradients)
- Procedure per iteration:
  1. Sort training samples by absolute gradient magnitude
  2. Retain the top $a\%$ of samples with largest gradients
  3. Randomly sample $b\%$ of the remaining low-gradient samples
  4. Scale up the low-gradient examples by factor $\frac{1-a}{b}$ in gradient/Hessian sums
- Reduces data size for split finding without significantly affecting accuracy
- Typical settings: $a = 40\%$, $b = 20\%$

GOSS Numeric Example

Absolute gradients (sorted): $[9, 8, 7, 6, 5, 4, 3, 2, 1, 0.5]$ Parameters: $a = 40\%$ (retain top 4), $b = 20\%$ (sample 2 of remaining 6)

Selected samples:

- High-gradient: indices 1–4 $(9, 8, 7, 6)$
- Low-gradient random sample: indices 5 and 7 $(5, 3)$

Scaling factor for low-gradient examples:

$$\frac{1 - 0.4}{0.2} = 3$$

Resulting gradients used for training:

$$[9,\ 8,\ 7,\ 6,\ 5 \times 3 = 15,\ 3 \times 3 = 9]$$

Exclusive Feature Bundling (EFB): Concept

- EFB reduces dimensionality by grouping mutually exclusive sparse features into a single bundle
- Ideal for one-hot encoded or sparse categorical variables that rarely co-occur
- Builds a compatibility graph: nodes = features, edges connect features that co-occur
- Uses a greedy algorithm to pack non-overlapping features into bundles
- Results in dramatically fewer features and lower memory footprint without information loss

EFB Numeric Example (with Offsets)

- Original sparse features are mutually exclusive; we pack them into bundle B_1.
- To maintain identifiability, we apply offsets to each feature's range.

Original Sparse Features:

Sample	f_1	f_2	f_3	f_4
1	8	0	0	0
2	0	5	0	0
3	0	0	3	0
4	0	0	0	9

Bundled Feature B_1 (Offsets: $f_1 = 0, f_2 = 10, f_3 = 20, f_4 = 30$):

Sample	B_1	Effective Split Logic
1	**8**	$B_1 \in [0, 10] \rightarrow$ acts as f_1
2	**15**	$B_1 \in [11, 20] \rightarrow$ acts as f_2
3	**23**	$B_1 \in [21, 30] \rightarrow$ acts as f_3
4	**39**	$B_1 \in [31, 40] \rightarrow$ acts as f_4

Handling Categorical Features in LightGBM

- LightGBM natively supports categorical features, eliminating the need for one-hot encoding.
- During split finding, each category is treated as a separate bin. The algorithm searches for the optimal binary partition (e.g., $\{A, B\}$ vs. $\{C\}$) that maximizes the split gain.
- Internally, categorical values are sorted by an aggregated statistic (such as the mean target value) to facilitate efficient split search.
- **Benefits:**
  - Reduced dimensionality and memory usage.
  - Faster training since the feature space remains compact.
  - Lower risk of overfitting compared to using one-hot encoding.

Categorical Split Example

Consider the following dataset:

Sample	Feature Cat	Target
1	A	10
2	B	12
3	A	11
4	C	4
5	B	13
6	C	5

LightGBM calculates the mean target for each category:

$$\text{Mean for A} = \frac{10+11}{2} = 10.5, \quad \text{Mean for B} = \frac{12+13}{2} = 12.5, \quad \text{Mean for C} = \frac{4+5}{2} = 4.5.$$

Sorting by mean gives the order: B (12.5), A (10.5), C (4.5). An optimal split is to group $\{A, B\}$ versus $\{C\}$, producing two groups with mean targets 11.5 and 4.5, respectively. This split maximizes the reduction in loss.

Missing-Value Handling

- Both LightGBM and XGBoost automatically treat missing values as a separate category
- At each split, they evaluate sending missing entries left or right and choose the direction with higher gain
- No manual imputation required — the algorithm learns the optimal "default direction" during training

LightGBM Training Algorithm (Part 1)

Initialize $F_0(x) = \arg\min_\gamma \sum_{i=1}^{N} L(y_i, \gamma)$
For $m = 1 \ldots M$:

1. Compute gradients g_i and Hessians h_i
2. Apply Gradient-Based One-Side Sampling (GOSS):
   - Retain top $a\%$ of examples by —gradient—
   - Randomly sample $b\%$ of remaining examples
   - Scale gradients of sampled examples by $\frac{1-a}{b}$
3. Apply Exclusive Feature Bundling (EFB) to compress sparse features
4. Build a leaf-wise tree $f_m(x)$:
   - Evaluate split gain for each candidate
   - Split the leaf with highest gain if gain $>$ `min_gain_to_split`
   - Stop when `num_leaves` reached or no valid split

LightGBM Training Algorithm (Part 2)

5. Compute optimal leaf weights:

$$w_j = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$$

6. Update ensemble prediction with learning rate η:

$$F_m(x) = F_{m-1}(x) + \eta \, f_m(x)$$

7. Repeat until M trees are built
8. **Return** final model $F_M(x)$

LightGBM Key Hyperparameters & Tuning

Parameter	Purpose	Typical Range / Guidance
num_leaves	Maximum leaves per tree; controls complexity	31–255
learning_rate	Step size shrinkage; slower learning reduces overfitting	0.01–0.2
n_estimators	Number of trees to build	100–1000 (use early stopping)
max_depth	Limits tree depth; prevents overly deep branches	3–7
min_data_in_leaf	Minimum samples required in a leaf	10–50
feature_fraction	Fraction of features used per tree (column subsample)	0.5–0.8
bagging_fraction	Fraction of data used per iteration (row subsample)	0.6–0.8
bagging_freq	Frequency of bagging (0 = no bagging)	1–5
min_gain_to_split	Minimum split gain required to make a split	0–5
lambda	L2 regularization on leaf weights	0–10

Comparison: XGBoost vs LightGBM

Characteristic	**XGBoost**	**LightGBM**
Tree Growth Strategy	Level-wise (balanced splits)	Leaf-wise (best-first splits)
Missing Values	Sparse-aware split direction	Sparse-aware split direction
Categorical Handling	Requires encoding	Native handling (optimal grouping)
Sampling	Row/column subsample only	GOSS + EFB + subsampling
Split Gain Pruning	γ threshold	γ threshold + num_leaves
Memory Efficiency	Moderate	High (feature bundling reduces dimension)
Training Speed	Fast with parallelization	Faster (leaf-wise + sampling)
GPU Support	Yes	Yes
Key Hyperparameters	eta, max_depth, subsample, colsample, gamma, lambda	learning_rate, num_leaves, feature_fraction, bagging_fraction, min_data_in_leaf
Typical Use Cases	Balanced accuracy, stability	Very large datasets, sparse features

Coding Part 2

Summary

Lecture 11 Summary: Gradient Boosting & Gradient-Boosted Trees

- **Gradient Boosting**: sequentially adds weak learners to correct residuals via functional gradient descent
- Core algorithm: compute pseudo-residuals $\rightarrow$ fit tree $\rightarrow$ line search for global step size $\rightarrow$ update ensemble
- **Enhancements in GBT**:
  - Region-specific step sizes for finer updates
  - Shrinkage (learning rate) to slow learning and reduce overfitting
  - Row & column subsampling for variance reduction
  - Tree structure constraints (num_leaves, max_depth, min_data_in_leaf)
- Achieves powerful bias reduction with controlled variance $\rightarrow$ high predictive accuracy on tabular data

Advances Introduced by XGBoost

- **Regularized Objective**: integrates split complexity (γ) and leaf-weight (λ) penalties for stronger overfitting control
- **Second-Order Optimization**: uses both gradients and Hessians for more accurate, faster convergence
- **Sparse-Aware Splits**: automatically learns default direction for missing values
- **Cache-Efficient Block Structure**: compressed data blocks minimize cache misses and I/O overhead
- **Parallel and Distributed Training**: multi-threaded histogram construction and AllReduce for large-scale data
- **GPU Support**: hardware acceleration for dramatic speedups on very large datasets

Additional Innovations in LightGBM

- **Leaf-Wise (Best-First) Growth**: deeper splits on highest-gain leaves for faster loss reduction
- **Gradient-Based One-Side Sampling (GOSS)**: retains high-gradient examples, subsamples low-gradient ones to speed training without losing accuracy
- **Exclusive Feature Bundling (EFB)**: combines mutually exclusive sparse features into single bundles, reducing dimensionality by up to 80%
- **Native Categorical Support**: optimal categorical splits without manual encoding
- **Highly Scalable**: superior memory efficiency and speed (2×–5× faster than XGBoost on large sparse data)
- **Robust Regularization Controls**: num_leaves, min_data_in_leaf, and min_gain_to_split ensure effective overfitting prevention

Thank You!

Questions are welcome.